Laboratory 4:

Signals and Systems
LTI Systems & The Convolution Machine

Hardware-in-the-Loop Discrete-Time Convolution

Lab Duration: 2 hours (time-critical with skeleton code provided)
Pre-Lab Time Expected: 40 minutes (hand convolution, sequence assignment, code review)
Key Concept: Transform abstract convolution mathematics into observable real-time hardware output and MATLAB verification; understand FIR filters as impulse responses.

Contents

About This Lab Manual

Note on Media Placeholders: This lab manual contains boxed placeholders labeled "[INSERT ...]" that indicate where professional reference media should be embedded by instructors. These are **NOT** spaces for students to submit content. Examples include:

- **Reference circuit schematics** - showing correct hardware wiring configurations
- **Example oscilloscope screenshots** - demonstrating expected waveforms and measurements
- **Example MATLAB plots** - showing typical analysis outputs
- **Theoretical diagrams and block diagrams** - pedagogical reference material

Student-submitted materials (circuit photographs, measured data, MATLAB code outputs) are explicitly requested in **Deliverable** and **Checkpoint** sections and must include your name and student ID.

Grading and Authenticity Requirements

Deliverable (1/8): This is a graded laboratory assignment. Include your **name and student ID** in all required screenshots and photographs (oscilloscope, MATLAB plots, breadboard/setup photos).

Deliverable (2/8): All MATLAB convolution verification must use data captured from your own hardware setup. Results computed entirely in MATLAB without Arduino acquisition are not accepted as hardware verification evidence.

Checkpoint (1/5): Before submission, verify that all screenshots are legible, include axis labels with units and title information, and display your identifying information clearly.

Title & Real-World Context

You are a Junior DSP Engineer at a High-End Audio Effects Manufacturer

Your technical lead has tasked you with designing and validating the core convolution engine for a **premium real-time audio effects processor** deployed in professional music production studios and concert sound systems worldwide. The system must reliably deliver three demanding use cases:

1. **Impulse Response Convolution Reverb (Flagship Feature).** Your company's signature product is a convolution reverb engine that simulates the acoustics of famous concert halls, vintage recording studios, and cathedral spaces by convolving live audio with carefully captured impulse responses (IRs). Each IR is a FIR filter with 4,000–48,000 taps, depending on the venue. The convolution must be **mathematically exact**—even a single computational error or off-by-one indexing bug will cause artifacts ("grittiness," frequency coloration, phase distortion) that professional audio engineers immediately detect. Your validation workflow tests the convolution algorithm in real time against MATLAB reference implementations to guarantee bit-exact accuracy before deployment.

2. **Custom Impulse Response Design and Swappability.** Studio customers capture IR measurements from unique spaces (clubs, theaters, home studios) and upload them as finite-length sequences to the processor. The system must correctly convolve arbitrary-length IRs (from 512 taps to 64k taps) with incoming audio, automatically computing output lengths and managing buffer overflow without introducing latency spikes or ringing. Time-varying IR parameters (reverb decay envelope, frequency-dependent absorption) are critical; your lab validates that the convolution operation preserves the intended filtering behavior across different IR families.

3. **Real-Time Performance and Latency Certification**. A professional music producer recording vocals in real time requires sub-20ms latency for the reverb effect; otherwise, the performer hears echo and loses timing synchronization with the backing track. The convolution algorithm must execute in a bounded time window and deliver deterministic output timing. You will measure and certify that your hardware convolution implementation meets this latency budget while maintaining numerical accuracy.

Your Mission. In this lab, you will build and validate a discrete-time convolution "machine" that demonstrates the core concepts underpinning production audio effects processors:

- Design a pair of input sequences ($x[n]$ and $h[n]$) using your TA-assigned parameters.
- Implement the convolution sum $y[n] = \sum_k x[k]\, h[n-k]$ in embedded C++ on Arduino.
- Verify hardware output against MATLAB's reference `conv()` function using live captured data.
- Produce publication-quality stem plots showing the flip–shift–multiply–sum interpretation.
- Animate the convolution process to internalize the kernel-sliding algorithm.

The audio engineering industry demands mathematical rigor: any convolution error, no matter how subtle, is immediately audible to trained ears. Your lab bridges the gap between abstract signal processing theory and production-grade real-time implementation. The skills you demonstrate here—exact algorithm implementation, rigorous verification against reference code, latency

analysis—are directly applicable to professional audio, radar, communications, and biomedical signal processing roles.

Constraints (Fixed). 2-hour lab execution window (time-critical; skeleton code provided to focus on convolution logic). Unique TA-assigned sequences ensure independent work. All infrastructure code provided; you write the convolution function body and MATLAB verification. All checkpoints and deliverables remain exactly as specified herein.

A Note on Skeleton Code and Time Management

This lab provides skeleton code for two tasks that would otherwise consume the entire 2-hour lab window. Download both files from Canvas before arriving at lab:

- `Lab04_Skeleton.ino` — Arduino sketch with `setup()`/`loop()` structure, `Serial.begin()`, LED output driver, and pin-mode initialization pre-written. You fill in the convolution function body, the DIP-switch read logic, and the pushbutton read logic.
- `Lab04_MATLAB_Skeleton.m` — MATLAB script with serial port initialization, raw-line reading via `readline()`, and ASCII string parser pre-written. You fill in the `conv()` verification call, the `stem()` subplot formatting, and the inner animation math.

The skeleton code is intentionally incomplete. You must write the core algorithmic content in both files. The infrastructure (serial handshaking, port configuration, LED indexing, string splitting) is provided so your 2-hour window is spent on convolution — not serial debugging.

Suggested time allocation:

- **0:00–0:15** Receive TA assignment, review pre-lab calculations, begin wiring.
- **0:15–0:40** Complete wiring, TA wiring check, flash skeleton sketch.
- **0:40–1:00** Implement convolution function; achieve CP-2 (hardware demo to TA).
- **1:00–1:40** MATLAB Tasks 1–3; record video; complete CP-3.
- **1:40–2:00** Final screenshots, clean up bench, TA sign-off on Table 5.

How This Manual Is Organized

- **Main sections (quick execution path):** what to build, what to run, and what to show at CP-1/CP-2/CP-3.
- **Appendix A–B:** optional encoding mode and troubleshooting.
- **Appendix C–D:** full Arduino and MATLAB reference templates moved out of the main flow for faster in-lab navigation.

1 Learning Objectives

Upon completion of this laboratory, you will demonstrate:

1. **Hand Computation of DT Convolution.** Use the flip–shift–multiply–sum procedure to compute $y[n] = \sum_k x[k]\, h[n-k]$ for all output indices, explicitly verify output length using $L_y = L_x + L_h - 1$, and validate your result against both hardware measurements and MATLAB.

2. **Embedded Implementation of the Convolution Sum.** Write efficient C++ code to implement convolution on an Arduino, managing finite-precision integer arithmetic, hardware I/O (DIP switches, pushbuttons, LED bar), serial communication, and deterministic execution within a real-time constraint.

3. **Hardware–Software Correlation and Error Analysis.** Acquire dual-channel oscilloscope and MATLAB serial streams, compare hardware output against MATLAB's reference `conv()` function, quantify numerical agreement (RMS error, boundary effects), and identify root causes of discrepancies (quantization, indexing, overflow).

4. **Visualization and Interpretation of Discrete Signals.** Create publication-quality stem plots with correct axis ranges, labels, and discrete-time indexing; interpret visual patterns in $x[n]$, $h[n]$, and $y[n]$ to identify the filtering action (low-pass, high-pass, resonance, or custom shaping).

5. **Animated Convolution Process and Intuitive Understanding.** Develop a frame-by-frame MATLAB animation that visualizes the flip–shift–multiply–sum process, showing how the flipped kernel sweeps across the input and accumulates products. Translate the animation into articulate explanation of the convolution mechanism.

6. **FIR Filter Characterization and Production-Grade Validation.** Analyze your TA-assigned impulse response $h[n]$ to determine its filtering intent (moving average, edge detector, or custom shape), predict its frequency-domain effect (low-pass, high-pass, notch, etc.), and validate predictions against measured hardware behavior.

2 Core Concepts: Discrete-Time Convolution and LTI Systems

2.1 LTI Systems and the Impulse Response

A system $\mathcal{T}\{\cdot\}$ is Linear and Time-Invariant (LTI) if it satisfies two properties simultaneously:

- **Linearity (Superposition):**

$$\mathcal{T}\{a\,x_1[n] + b\,x_2[n]\} = a\,\mathcal{T}\{x_1[n]\} + b\,\mathcal{T}\{x_2[n]\}$$

  for all scalars a, b and admissible inputs x_1, x_2.

- **Time Invariance:** If $y[n] = \mathcal{T}\{x[n]\}$, then $\mathcal{T}\{x[n - n_0]\} = y[n - n_0]$ for every integer shift n_0.

The power of LTI theory lies in a remarkable fact: an LTI system is **completely characterized** by its impulse response $h[n]$, defined as the system output when the input is the unit impulse $\delta[n]$:

$$h[n] \triangleq \mathcal{T}\{\delta[n]\}.$$

Once $h[n]$ is known, the response to any input $x[n]$ can be determined without re-analyzing the system's internals—this is one of the most powerful tools in all of signals and systems.

2.2 Discrete-Time Convolution

The input–output relationship for any LTI system is given by the **convolution sum**:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]\,h[n-k]. \tag{1}$$

For finite-support sequences—which is always the case in this lab and in virtually all real digital signal processing—the sum has finitely many nonzero terms. If $x[n]$ is nonzero for $n = 0, 1, \ldots, L_x - 1$ (length L_x) and $h[n]$ is nonzero for $n = 0, 1, \ldots, L_h - 1$ (length L_h), then $y[n]$ is nonzero for $n = 0, 1, \ldots, L_y - 1$ where:

$$L_y = L_x + L_h - 1. \tag{2}$$

This length formula is one of the most useful facts you will internalize in this course. Verify your hand calculations against it before touching the hardware.

2.3 The Flip–Shift–Multiply–Sum Interpretation

Equation (1) has a beautiful graphical interpretation that gives convolution its intuitive meaning. To compute a single output sample $y[n_0]$ for a fixed index n_0:

1. **Flip:** Reflect $h[k]$ about the vertical axis to obtain $h[-k]$.
2. **Shift:** Slide the flipped sequence right by n_0 samples to obtain $h[n_0 - k]$.
3. **Multiply:** Compute the element-wise product $x[k] \cdot h[n_0 - k]$ for all k.
4. **Sum:** Add all products to obtain the scalar $y[n_0] = \sum_k x[k]\,h[n_0 - k]$.

Repeating this procedure for every integer n_0 in the output support sweeps $h[-k]$ from left to right across $x[k]$. This sliding-kernel picture is exactly what the MATLAB animation in Phase 3 will bring to life.

2.4 Output Length and Support

Table 1 summarizes how the support of $y[n]$ is determined from those of $x[n]$ and $h[n]$.

Table 1. Support and Length of Convolution Output

Fact	**Details**
Output length	$L_y = L_x + L_h - 1$; always longer than either input alone.
Start of support	$n_{\min}$ = start of $x[n]$ + start of $h[n]$ (both 0 in this lab).
End of support	$n_{\max} = n_{\min} + L_y - 1$.
First/last frames	The first frame ($n_0 = 0$) and last frame ($n_0 = L_y - 1$) have only one nonzero term in the sum; maximum-overlap frames are in the middle.
LED display limit	This circuit displays indices $n = 0$ through 6 (7 LEDs). The overflow LED activates when $L_y > 7$.

2.5 Convolution Properties

Three properties will appear implicitly in your hardware experiment:

- **Commutativity:** $x[n] * h[n] = h[n] * x[n]$. You could swap the roles of DIP switches and pushbuttons without changing the numerical output.
- **Associativity:** $(x * g) * h = x * (g * h)$. Two LTI systems in cascade can be combined into one by convolving their impulse responses.
- **Distributivity over Addition:** $x * (g + h) = x * g + x * h$. Two LTI systems in parallel can be combined by adding their impulse responses.

2.6 Convolution as a Finite Impulse Response (FIR) Filter

Every finite-length $h[n]$ defines an FIR digital filter. The shape of $h[n]$ determines the system's "character": a uniform $h[n] = \frac{1}{L}\{1, 1, \ldots, 1\}$ is a moving-average (low-pass) filter, while an alternating $h[n] = \{1, -1\}$ highlights rapid changes (high-pass / edge detector). Your TA-assigned $h[n]$ is designed to exhibit a recognizable filtering behavior; identifying it is part of your post-lab analysis.

3 Pre-Laboratory Assignment

Complete the pre-lab before arriving at lab. You will show your handwritten worksheet to the TA as Checkpoint CP-1 (target: first 15 minutes).

3.1 Part A: Receiving Your TA-Assigned Sequences

At the **start of the lab session**, your TA will assign a unique pair of sequences to your group. Record them immediately in Table 2.

Table 2. TA-Assigned Convolution Sequences — Record at Start of Lab

Symbol	Description	Assigned Values
$h[n]$	Impulse response, encoded on DIP switches	_____
$x[n]$	Input sequence, entered via pushbuttons	_____
L_h	Length of $h[n]$	_____
L_x	Length of $x[n]$	_____
$L_y = L_x + L_h - 1$	Expected output length	_____
Random Seed	Unique pair identifier	_____

Encoding note: Each DIP switch ON ($= 1$) or OFF ($= 0$) represents one sample of $h[n]$. Pushbuttons similarly produce binary entries for $x[n]$ (pressed $= 1$, released $= 0$). If your TA assigns non-binary coefficients (values 0–3), consult Appendix A for the two-switch encoding convention.

3.2 Part B: Hand Calculation Worksheet

Pre-lab Deliverable (1/2): Using your assigned $h[n]$ and $x[n]$, compute every output sample $y[n_0]$ by hand using the explicit flip–shift–multiply–sum procedure. You must show the four-step work (Flip $\rightarrow$ Shift $\rightarrow$ Multiply $\rightarrow$ Sum) individually for each $n_0 = 0, 1, \ldots, L_y - 1$. Answers without shown work receive zero credit.

Format to follow for each output sample:

For each n_0, write out:

1. The index axis k spanning all nonzero values of $x[k]$ and $h[n_0 - k]$.
2. The row of values $x[k]$ (fixed for all n_0, just re-copy it each time).
3. The row of values $h[n_0 - k]$ (the flipped-and-shifted kernel at the current position).
4. The row of element-wise products $x[k] \cdot h[n_0 - k]$.
5. The sum of those products, clearly boxed as $y[n_0] = \square$.

Worked example (not your actual sequences) — study this format carefully:

Let $h[n] = \{2, 1, 3\}$ for $n = 0, 1, 2$ and $x[n] = \{1, 0, 2\}$ for $n = 0, 1, 2$. Then $L_y = 3+3-1 = 5$.

Computing $y[2]$: Flip $h[k]$ to get $h[-k] = \{3, 1, 2\}$, then shift right by 2 to place $h[0]$ at $k = 2$:

k :	0	1	2	3	4
$x[k]$:	1	0	2	0	0
$h[2-k]$:	3	1	2	0	0
product:	3	0	4	0	0

$$y[2] = 3 + 0 + 4 = \boxed{7}$$

Repeat for $y[0], y[1], y[3], y[4]$ to complete the full output sequence.

Record all of your hand calculations in the box below. Your TA will review this at CP-1.

PRE-LAB WORKSHEET: Flip–Shift–Multiply–Sum hand calculations for all output samples $y[n_0],\ n_0 = 0, \ldots, L_y - 1$. Show the four-step table for each sample. Write your name and student ID at the top of this box.

3.3 Part C: Pre-Lab MATLAB Verification

Pre-lab Deliverable (2/2): Before arriving at lab, verify your hand calculation in MATLAB using the built-in `conv()` function. If you discover a discrepancy, correct your hand calculation and note the error source.

```
% Replace these example sequences with your TA-assigned values:
h = [2, 1, 3];       % h[n]  -- TA assigned
x = [1, 0, 2];       % x[n]  -- TA assigned
```

```
y_prelab = conv(x, h);
n_y      = 0 : length(y_prelab) - 1;

stem(n_y, y_prelab, 'filled');
xlabel('n'); ylabel('y[n]'); grid on;
title(['Pre-Lab Verification: y[n] = x[n] * h[n] | ', ...
       'YOUR NAME & STUDENT ID HERE']);
```

Deliverable (3/8): Submit a screenshot of your pre-lab `stem()` plot. Your name and student ID must appear in the `title()` string. Note in your report whether your hand calculation matched or disagreed with `conv()`, and if it disagreed, state which sample(s) were wrong and why.

Pre-Lab Deliverable: MATLAB conv() verification stem plot with name/student ID in title

Discussion (1/9): (Pre-lab question) If your hand calculation and MATLAB's `conv()` disagreed, identify the specific output index n_0 where the error occurred. Trace through the flip–shift–multiply–sum steps at that index to find the arithmetic mistake. Describe the error in two to three sentences.

4 Equipment & Setup

Table 3. Required Hardware and Software

Category	Required Items
Microcontroller	Arduino Mega 2560 (required for full pin map), USB Type-A to Type-B cable
$h[n]$ Input	8-position DIP switch bank (e.g., CTS Series 206, or equivalent slide-switch array)
$x[n]$ Input	4 momentary normally-open pushbuttons (through-hole breadboard type)
$y[n]$ Display	10-segment LED bar graph module (e.g., Kingbright DC-10EWA or equivalent)
Resistors	10 k$\Omega \times 12$ (pull-down resistors for DIP switches and buttons), 220 $\Omega \times 8$ (LED current-limiting resistors)
Prototyping	Full-size solderless breadboard, assorted jumper wires
Software	Arduino IDE (v1.8 or v2.x), MATLAB R2021a or later
Skeleton files	`Lab04_Skeleton.ino` and `Lab04_MATLAB_Skeleton.m` (download from Canvas before lab)

4.1 Circuit Overview

Figure 1 shows the signal flow of the Convolution Machine. The DIP switch bank supplies $h[n]$ coefficients on digital input pins D2–D9. Four pushbuttons supply $x[n]$ samples on pins D10–D13. The Arduino firmware computes $y[n] = x[n] * h[n]$ and drives the 10-segment LED bar graph to display the output. Simultaneously, the firmware formats an ASCII string and transmits it at 115200 baud over USB to MATLAB for software verification and animation.

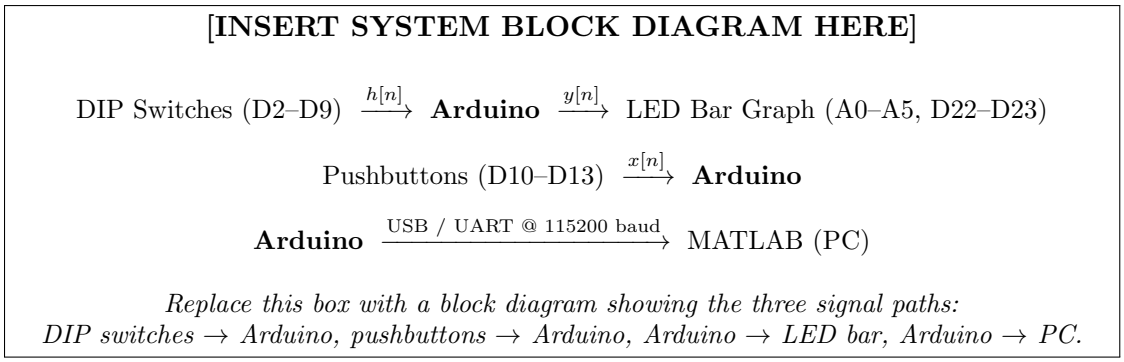

Figure 1. Convolution Machine system block diagram. Three data paths converge at the Arduino: two hardware inputs ($h[n]$ and $x[n]$) and two hardware outputs (LED bar graph and serial port).

4.2 Pin Assignment Reference

Table 4. Arduino Pin Assignments for the Convolution Machine

Signal	**Arduino Pin(s)**	**Direction**	**Notes**
$h[0]$–$h[7]$	D2–D9	INPUT	DIP switches; 10 kΩ pull-down to GND
$x[0]$–$x[3]$	D10–D13	INPUT	Pushbuttons; 10 kΩ pull-down to GND
$y[0]$–$y[5]$	A0–A5	OUTPUT	LED bar segments 1–6 via 220 Ω series resistors
$y[6]$	D22	OUTPUT	LED bar segment 7 via 220 Ω series resistor
Overflow LED	D23	OUTPUT	Illuminates if $L_y > 7$ (output exceeds display range)
Serial TX	USB (built-in)	OUTPUT	115200 baud, 8N1, CR/LF line terminator

Board note: This pin map requires an Arduino Mega so that LED outputs use dedicated pins (D22 and D23) and do not conflict with DIP inputs. If your section uses an Uno, use the TA-provided reduced-pin variant and corresponding skeleton constants.

4.3 Step-by-Step Wiring Instructions

Work through the four wiring steps in order. Do not power the Arduino until Step W4 is complete and your TA has visually verified the circuit.

4.3.1 Step W1: DIP Switch Bank ($h[n]$ Inputs)

1. Orient the 8-position DIP switch bank across the center gap of the breadboard (one row of pins on each side of the gap).
2. Connect the **common rail** side (all left-hand pins) of the DIP switch to the breadboard's +5 V power rail.
3. From the output side (right-hand pins, positions 1–8), run one jumper wire to each Arduino pin D2 through D9 respectively (switch position 1 → D2, position 2 → D3, . . . , position 8 → D9).
4. Install a 10 kΩ pull-down resistor from each Arduino input pin (D2–D9) to GND. This ensures the pin reads a clean logic LOW when the switch is open; without pull-downs, the pin floats and reads unpredictably.

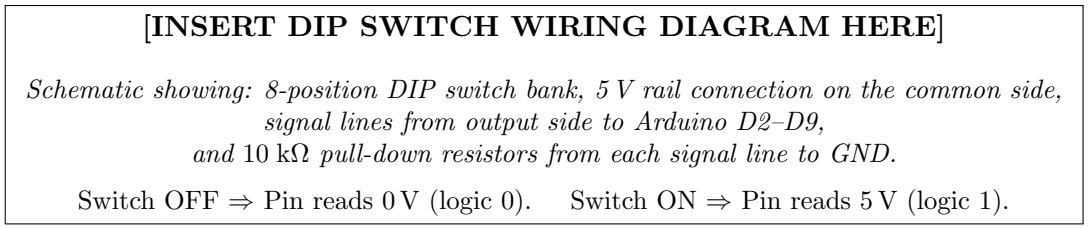

Figure 2. DIP switch bank wiring for $h[n]$ coefficient input. Each switch position encodes one sample of $h[n]$: ON = 1, OFF = 0.

4.3.2 Step W2: Pushbutton Inputs ($x[n]$ Inputs)

1. Place 4 momentary pushbuttons in a row on the breadboard with at least one empty column between buttons to avoid accidental bridging.
2. Connect one terminal of each button to the +5 V rail.
3. Connect the other terminal of each button to Arduino pins D10, D11, D12, and D13 (for $x[0]$, $x[1]$, $x[2]$, $x[3]$ respectively).
4. Install a 10 kΩ pull-down resistor from each signal line (D10–D13) to GND.
5. **Label each button** with a small piece of tape: "$x[0]$", "$x[1]$", "$x[2]$", "$x[3]$". Correct labeling prevents input errors during the live demonstration.

[INSERT PUSHBUTTON WIRING DIAGRAM HERE]

Schematic showing: 4 normally-open pushbuttons, 5 V supply on one terminal, signal lines from the other terminal to Arduino D10–D13, and 10 kΩ pull-down resistors from each signal line to GND.

Button released ⇒ D10–D13 read 0 V (logic 0). Button pressed ⇒ reads 5 V (logic 1).

Figure 3. Pushbutton wiring for $x[n]$ sample input. Pressing button k holds $x[k] = 1$ for the duration of the press; releasing returns it to 0.

4.3.3 Step W3: LED Bar Graph Output ($y[n]$ Display)

1. Place the 10-segment LED bar graph module on the breadboard. Check the datasheet (or the label on the module) to identify the anode (positive) and cathode (negative/common) pins.
2. Connect all cathode pins to GND.
3. Place a 220 Ω current-limiting resistor in series with each of the 7 active anode pins (LED segments 1–7).
4. Connect the free end of each 220 Ω resistor to the Arduino output pins as specified in Table 4: segment 1 → A0, segment 2 → A1, . . . , segment 6 → A5, segment 7 → D22.
5. Connect LED segment 8 (the overflow indicator) to Arduino D23 through its own 220 Ω resistor.
6. Leave LED segments 9–10 unconnected.

[INSERT LED BAR GRAPH WIRING DIAGRAM HERE]

Schematic showing: 10-segment LED bar graph, shared cathode to GND, 220 Ω series resistors on segments 1–8, connections to Arduino A0–A5 (segments 1–6), D22 (segment 7), D23 (overflow).

LED lit ⇒ $y[n] \neq 0$ (non-zero output sample). LED off ⇒ $y[n] = 0$.

Figure 4. LED bar graph wiring for $y[n]$ output display. Segments 1–7 represent output samples $y[0]$–$y[6]$. The overflow LED (segment 8) indicates that the output sequence extends beyond 7 samples.

4.3.4 Step W4: Pre-Power Verification Checklist

Before connecting the Arduino to USB, perform this checklist:

☐ **DIP pull-downs:** 8 resistors present, one per switch, each running from the Arduino input pin to GND.

☐ **Button pull-downs:** 4 resistors present, one per button, each running from the Arduino input pin to GND.

- ☐ **LED series resistors:** 8 resistors present (segments 1–7 and overflow). **No LED anode is directly connected to 5 V or to an Arduino output without a series resistor** — this will burn out the LED.
- ☐ **No 5 V to GND short:** No jumper wire directly connecting the power rail to the GND rail.
- ☐ **Total resistor count:** $8 + 4 + 8 = 20$ resistors on the breadboard.

Checkpoint (2/5): Show your completed, unpowered circuit to your TA before plugging in the USB cable. The TA will visually verify the pull-down network and LED resistor chain. Photo the assembled circuit with your student ID visible in the frame.

Checkpoint W (Wiring): TA verifies circuit before power-on. Photograph of assembled circuit with student ID card visible in frame.

[INSERT COMPLETE ASSEMBLED CIRCUIT PHOTOGRAPH HERE]

Full breadboard photo showing: DIP switch bank (top-left), pushbutton row (center), LED bar graph (top-right), Arduino (right), and all wiring. Student ID card visible.

Use this reference photo to verify your own assembly matches the expected layout.

Figure 5. Reference photograph of the complete Convolution Machine circuit. Your circuit should closely match this layout before powering on.

5 Software Implementation — Arduino (Phase 2)

Open `Lab04_Skeleton.ino` from Canvas. The skeleton provides `setup()` with all `pinMode()` calls, `Serial.begin(115200)`, and the LED driver function `displayOutput()`. Your task is to implement the four clearly marked `// --- YOUR CODE HERE ---` regions:

1. `readDIPSwitches()` — fills the `h[]` array from pins D2–D9.
2. `readPushbuttons()` — fills the `x_in[]` array from pins D10–D13.
3. `computeConvolution()` — implements the nested-loop convolution sum.
4. The serial output formatter in `loop()`.

5.1 Reading the DIP Switches and Pushbuttons

Implement two short loops:

- `readDIPSwitches()`: read D2–D9 into `h[0..7]`.
- `readPushbuttons()`: read D10–D13 into `x_in[0..3]`.

Use direct `digitalRead()` values (0/1) and preserve index-to-pin ordering. Full reference code is in Appendix C.

5.2 Implementing the Convolution Sum in C++

The convolution sum in Equation (1) translates directly into two nested `for` loops. The outer loop iterates over output index n from 0 to $L_y - 1$; the inner loop iterates over input index k from 0 to $L_x - 1$, accumulating $x[k] \cdot h[n-k]$ whenever `0 <= n-k < lh_actual`. Keep this boundary check exactly. Full template code is in Appendix C.

Discussion (2/9): Explain why the boundary check `0 <= n-k < lh_actual` is necessary. What would happen mathematically if you omitted it and accessed `h[n-k]` for $n - k < 0$? What does C++ actually do when you read an array out of bounds? Connect your answer to the physical meaning of a causal impulse response (one defined only for $n \geq 0$).

5.3 Displaying $y[n]$ on the LED Bar Graph

The LED driver in the skeleton lights segment n if $y[n] > 0$ and turns it off if $y[n] = 0$. The overflow LED on D23 should light if any nonzero sample appears beyond index 6. If overflow is lit, use MATLAB for full-vector verification. Reference implementation remains in Appendix C.

5.4 Serial Output to MATLAB

After computing and displaying $y[n]$, the Arduino must transmit one line in this exact schema:

```
H:h0,h1,...;X:x0,x1,...;Y:y0,y1,...
```

Keep labels `H:`, `X:`, `Y:` and semicolon delimiters unchanged so the MATLAB parser works. Full serial-format function template is in Appendix C.

Checkpoint (3/5): Flash the completed Arduino sketch. Open the Serial Monitor at 115200 baud. Flip DIP switches and press buttons while watching the output lines. Verify: (1) the line format matches the reference exactly, (2) the $h[n]$ field changes when you toggle DIP switches, and (3) the $x[n]$ field changes when you press/release buttons.

Checkpoint A (Arduino Serial): Serial Monitor screenshot showing correctly formatted output lines with live DIP/button updates. Student ID visible in the Serial Monitor window title or typed in the send field.

Checkpoint (4/5): With your TA-assigned sequences set on the DIP switches and pushbuttons, confirm that the LED bar graph displays the same nonzero pattern as your hand-calculated $y[n]$ from the Pre-Lab. Show this live to your TA for CP-2 sign-off.

Checkpoint CP-2 (Hardware Demo): Photo or video clip showing LED bar graph with TA-assigned sequences applied. LED pattern must match pre-lab hand calculation. TA initials obtained on Table 5.

6 Software Implementation — MATLAB (Phase 3)

Open `Lab04_MATLAB_Skeleton.m` from Canvas. The skeleton handles serial port initialization, raw-line reading via `readline()`, and the ASCII string parser. Implement the three tasks in the `% --- YOUR CODE HERE ---` sections.

6.1 Connecting to the Arduino

Set `port` for your OS, open `serialport(..., 115200)`, and parse one line into `h_hw`, `x_hw`, and `y_hw`. Close Arduino Serial Monitor first so MATLAB can claim the port. Full parser reference is in Appendix D.

6.2 Task 1 — MATLAB Verification with `conv()`

Using the parsed `h_hw` and `x_hw` vectors received from the Arduino, compute the convolution independently in MATLAB and compare it to the Arduino's computed result `y_hw`.

Implement exactly these checks:

- `y_matlab = conv(x_hw, h_hw);`
- `max_err = max(abs(y_hw - y_matlab));`
- Print a PASS/FAIL report with vectors and `max_err`.

Full Task 1 code template is in Appendix D.

Deliverable (4/8): Submit a MATLAB Command Window screenshot showing the complete Task 1 verification report. Your name and student ID must be visible (type them in the Command Window prompt, or add a `fprintf` header line with your name). If disagreements exist, explain which sample(s) differed and propose a hardware or firmware root cause.

Task 1 Evidence: MATLAB Command Window showing conv() verification report with max-abs-error result and PASS/FAIL statement

Discussion (3/9): If `y_hw` and `y_matlab` disagree, identify at least three distinct causes. Consider: DIP switch contact bounce, floating input pins (missing pull-down), integer overflow in the C++ `int` accumulator, and Serial transmission errors. Classify each as a systematic error (same sample always wrong) or a sporadic error (mismatch varies between reads).

6.3 Task 2 — Stem Plots: $x[n]$, $h[n]$, and $y[n]$

Create a single MATLAB figure containing a 1×3 array of `stem()` subplots. This figure is the primary visual record of your lab experiment.

Build a single `1x3` figure with:

- Subplot 1: `x[n]` stem plot with labels and grid.
- Subplot 2: `h[n]` stem plot with labels and grid.
- Subplot 3: overlaid `y_matlab` and `y_hw` with legend.

Add name and student ID in `sgtitle(...)`. Full Task 2 template is in Appendix D.

Deliverable (5/8): Submit the completed 1×3 stem subplot figure. Requirements: name/student ID in `sgtitle`; all three subplots have labeled `'n'` x-axes and descriptive y-axis labels; grid enabled on all subplots; the $y[n]$ subplot overlays both hardware and MATLAB results with a legend.

Task 2 Evidence: 1×3 **MATLAB stem subplot figure showing x[n], h[n], and y[n] (hardware vs. MATLAB overlay). Student name/ID in the figure title.**

[INSERT SCREENSHOT OF COMPLETED MATLAB STEM SUBPLOT FIGURE HERE]

Expected layout: three side-by-side stem plots for $x[n]$ (blue), $h[n]$ (blue), and $y[n]$ (blue + red overlay).
Figure title reads: "Convolution Machine — Lab 4 — [Name] — ID: [ID]".
$y[n]$ subplot legend distinguishes MATLAB conv() (blue) from Arduino hardware (red dashed).

Figure 6. Target output for Task 2. Your completed figure should match this general layout. Use your actual TA-assigned sequences, not the example values shown here.

6.4 Task 3 — Animating the Flip–Shift–Multiply–Sum

This task builds a frame-by-frame animation that shows $h[-k]$ (the flipped kernel) sliding across $x[k]$ and accumulating the output $y[n]$ one sample at a time. The animation skeleton provides the figure window setup and the loop structure; you implement the mathematical content inside the loop.

6.4.1 What Each Animation Frame Must Show

For output index $n_0 = 0, 1, \ldots, L_y - 1$, each frame must display:

1. **Top subplot:** $x[k]$ as fixed blue stems; $h[n_0 - k]$ as red stems at the current shift position;

a text annotation showing $y[n_0] =$ (the current output value).

2. **Bottom subplot:** The growing output sequence $y[0], y[1], \ldots, y[n_0]$ as green stems, extending by one new sample per frame.

Implementation checklist for each frame n_0:

- Build shifted kernel samples `h[n0-k]` on a common `k_axis`.
- Top subplot: overlay fixed `x[k]` and shifted kernel with frame title showing `y[n0]`.
- Bottom subplot: plot accumulated output `y[0:n0]`.

Full Task 3 animation template is in Appendix D.

[INSERT SCREENSHOT OF MATLAB ANIMATION — MIDDLE FRAME ($n_0 = \lceil L_y/2 \rceil$) HERE]

Top subplot: $x[k]$ (blue stems, fixed) overlaid with $h[n_0 - k]$ (red stems, shifted kernel at current position).
Bottom subplot: accumulated $y[n]$ (green stems) for $n = 0$ through n_0.
Frame title reads: "Frame $n_0 = X$: $y[X] =$[value]".
The red kernel is clearly positioned at the middle of its sweep across $x[k]$.

Figure 7. Target layout for a single animation frame at the midpoint of the sweep. Capture this frame as a screenshot for your report.

Deliverable (6/8): Record a continuous screen-capture video of the full animation running for your TA-assigned sequences. The video must show: (1) the red $h[n_0 - k]$ kernel visibly shifting one position right between frames, (2) the green $y[n]$ output growing by exactly one new sample per frame, and (3) the final frame displaying a complete output sequence that numerically matches your hand calculation and your hardware LED pattern.

Deliverable (7/8): Capture a static screenshot of the animation at frame $n_0 = \lceil L_y/2 \rceil$ (the middle frame) and include it in your written report. Label the frame in a title or annotation so the grader can identify which n_0 it represents.

Discussion (4/9): In the animation, identify the frame n_0^* that produces the largest output sample $y[n_0^*]$ (the "maximum energy" frame). Explain in terms of the overlap area between $x[k]$ and $h[n_0^* - k]$ why this frame maximizes the product sum. Briefly distinguish DT convolution from DT cross-correlation: in what single step do they differ, and how would the animation change if you were animating cross-correlation instead of convolution?

7 Discussion and Analysis

Answer all discussion questions in your post-lab report. Each response should be 3–6 sentences supported by numerical evidence from your hardware measurements and MATLAB output.

Discussion (5/9): Compare your pre-lab hand-calculated $y[n]$ to (a) the LED bar graph pattern observed on the hardware and (b) MATLAB's `conv()` output. Report all output sample values numerically. If any discrepancy exists between any two of the three results, identify the specific index, the magnitude of the error, and a plausible root cause.

Discussion (6/9): The output length $L_y = L_x + L_h - 1$ is always strictly greater than both L_x and L_h (when both are ≥ 2). Explain intuitively why the output is longer than either input alone. In terms of the animation frames: which frames have only a partial overlap between $x[k]$ and the shifted kernel (the "ramp-up" and "ramp-down" zones), and which frames have full overlap?

Discussion (7/9): The Arduino convolution function uses C++ `int` arithmetic (typically 16-bit on Uno, 32-bit on Mega). Construct a specific example: give values of $h[n]$, $x[n]$, their lengths, and the worst-case product term that could overflow a 16-bit signed integer (max = 32767). Propose a firmware-level fix using `long` or `int32_t`, and explain why integer overflow is an especially dangerous bug in embedded systems (hint: it is silent and non-exception-throwing in C++).

Discussion (8/9): Convolution is commutative: $x[n] * h[n] = h[n] * x[n]$. This means you could swap the sequences—set your assigned $x[n]$ on the DIP switches and enter $h[n]$ via pushbuttons. Verify this claim numerically for your assigned sequences by computing `conv(h_hw, x_hw)` in MATLAB and comparing it to `conv(x_hw, h_hw)`. Do the two results agree exactly? Would the LED bar graph display an identical or merely equivalent pattern? Note any indexing differences carefully.

Discussion (9/9): Examine the shape of your assigned $h[n]$. Does it resemble a moving-average filter (all positive coefficients of similar magnitude), a high-pass filter (alternating signs), an edge-detection kernel (e.g., $\{-1, 1\}$), or something else? Predict in words how this $h[n]$ would modify a long, smoothly varying input signal. What would happen to a rapidly oscillating input? (You do not need hardware data to answer this—use your understanding of the overlap-and-accumulate picture from the animation.)

8 Deliverables & TA Sign-Off Sheet

8.1 In-Lab Checkpoints

Complete all three checkpoints **during the lab session**. Your TA must initial Table 5 for the hardware checkpoints to count toward your grade.

Table 5. TA Checkpoint Sign-Off Sheet — Keep This Page and Submit a Photograph of It

CP		Requirement for Sign-Off	Target Time	TA Initials
CP-1	Pre-Lab	Hand-written flip–shift–multiply–sum worksheet for all L_y output samples, shown open to the TA. Four-step format (flip, shift, multiply, sum) visible for each sample. Values match the TA-assigned sequences. Calculator-only or answer-only pages do not qualify.	0:00–0:15	_____
CP-2	Hardware Demo	Powered Arduino with the assigned DIP switch configuration and pushbutton states: LED bar graph displays the correct $y[n]$ pattern. Serial Monitor (or TA-observed screen) shows correctly formatted output lines. TA compares LED pattern to CP-1 hand calculation.	0:40–1:00	_____
CP-3	MATLAB Demo	MATLAB script running live: (a) Command Window shows Task 1 PASS/FAIL report; (b) 1×3 stem subplot figure is open and complete; (c) animation runs at least one full pass from $n_0 = 0$ to $n_0 = L_y - 1$ with the kernel visibly sliding.	1:00–1:45	_____

Group member name 1: _____

Group member name 2: _____

TA name: _____ **Lab section / Date:** _____

8.2 Video Recording Requirements

Checkpoint (5/5): Before leaving lab, record a single continuous screen-and-camera video (maximum 4 minutes). It must include, in order:

1. A clear close-up of the DIP switch bank with your assigned $h[n]$ configuration visible; state aloud which switches are ON.
2. A clear close-up of the pushbutton row with your assigned $x[n]$ configuration held; state aloud

which buttons are pressed.

3. The LED bar graph fully visible, showing the resulting $y[n]$ pattern.
4. Switch to the MATLAB window and pan across the completed 1×3 stem subplot figure.
5. The animation running from frame 0 to the final frame (speed up if needed, but all frames must be visible).
6. Briefly hold your student ID card toward the camera.

Deliverable (8/8): Upload the video file (MP4 or MOV, max 500 MB) to Canvas using the filename format `Lab04_[LastName]_[StudentID].mp4`. Late video submissions without prior TA approval will be penalized.

8.3 Written Report Submission Checklist

- ☐ Pre-lab hand calculation worksheet (scanned or photographed, name and student ID visible at the top).
- ☐ Pre-lab MATLAB `conv()` stem plot with name/student ID in the plot title.
- ☐ Pre-lab discussion answer: did your hand calculation match `conv()`? If not, what was the error?
- ☐ Circuit photograph with student ID visible (taken at CP-2 sign-off).
- ☐ Serial Monitor screenshot confirming correct formatted output (CP-2 evidence).
- ☐ Task 1: Command Window screenshot showing max-abs-error verification report (PASS or FAIL).
- ☐ Task 2: 1×3 stem subplot figure with name and student ID in `sgtitle`.
- ☐ Task 3: Animation middle-frame screenshot (frame $n_0 = \lceil L_y/2 \rceil$) with frame number labeled.
- ☐ Video uploaded to Canvas separately (not embedded in the report PDF).
- ☐ Answers to all five **Discussion** questions (one paragraph each, with numerical evidence).
- ☐ Photograph of the signed Table 5 (TA must have initialed all three CP rows).
- ☐ Arduino `.ino` source code with your added functions in an Appendix.
- ☐ MATLAB `.m` source code (completed skeleton) in an Appendix.

9 Grading Rubric (100 Points)

Table 6. Lab 04 Evaluation Rubric

Category	**Points**	**Criteria**
Pre-lab hand calculation	20	All L_y output samples computed with explicit four-step flip–shift–multiply–sum work. Arithmetic correct. Full table format shown for each sample. TA CP-1 initials present.
Pre-lab MATLAB verification	5	Stem plot submitted with name/ID in title. Agreement or discrepancy with hand calculation noted; discrepancies diagnosed.
Hardware implementation (CP-2)	20	Functional circuit, TA CP-2 initials present. LED pattern matches correct convolution for assigned sequences. Serial Monitor screenshot in correct format. Circuit photograph with student ID.
Task 1 — `conv()` verification	10	Verification script correctly implemented. PASS/FAIL report printed with max-abs-error. Any discrepancy diagnosed in the report with a plausible root cause.
Task 2 — Stem subplot figure	15	All three subplots present, labeled axes, grid enabled, student name/ID in `sgtitle`. $y[n]$ subplot overlays hardware and MATLAB results with legend. Both results visible and distinguishable.
Task 3 — Animation & video	15	Animation implements correct flip, shift, and accumulation logic. Red kernel visibly slides right each frame. Green output grows one sample per frame. Video meets all six checklist requirements. Middle-frame screenshot submitted.
Discussion questions	15	All five questions answered in 3–6 sentences each, supported by numerical values from lab data. Technically precise; no qualitative-only responses.
Total	**100**	

Appendix A: Two-Switch Coefficient Encoding

If your TA assigns $h[n]$ coefficients with values in $\{0, 1, 2, 3\}$ (richer than the standard binary $\{0, 1\}$), the DIP switch bank encodes each coefficient using consecutive pairs of switches in simple binary. Table 7 shows the mapping; the extended skeleton's `readDIPSwitches()` function already handles this decoding when the macro `TWO_SWITCH_MODE` is defined.

Table 7. Two-Switch Binary Encoding for Coefficients 0–3

Coefficient Value	SW_A (MSB)	SW_B (LSB)
0	OFF	OFF
1	OFF	ON
2	ON	OFF
3	ON	ON

With 8 DIP switches grouped into 4 pairs, this encoding supports a length-4 $h[n]$ with values 0–3. If your TA assigns a length-8 binary $h[n]$, all 8 switches are used in single-switch mode (no pairing required).

Appendix B: Troubleshooting Guide

- **All LEDs stay ON regardless of DIP switch position.** Missing pull-down resistors cause input pins to float at an undefined voltage, which the Arduino often reads as logic 1. Measure the voltage at each input pin with a multimeter: it should read 0 V (switch OFF) or 5 V (switch ON). Install the 10 kΩ pull-downs if missing.
- **Serial Monitor shows garbled characters.** Verify that both the Arduino `Serial.begin()` call and the Serial Monitor baud-rate dropdown are set to exactly `115200`. Also confirm your USB cable is a data cable, not a charge-only cable (charge-only cables carry no data signals).
- **MATLAB throws `serialport: cannot open port.`** Only one application can hold the serial port at a time. Close the Arduino IDE Serial Monitor window before running the MATLAB script. On macOS, run `ls /dev/cu.*` in Terminal to identify the current port name, which changes each time the Arduino is re-plugged.
- **`y_hw` and `y_matlab` disagree by exactly ±1 for one sample.** Intermittent DIP switch contact is the most common cause. Toggle the suspect switch fully to its ON or OFF position and re-read. Worn breadboard contacts can also produce a logic level between 0 and 5 V, which the Arduino may read inconsistently.
- **The animation subplot is blank or shows only one sample.** Verify that `y_matlab` is populated before entering the loop (type `disp(y_matlab)` to check). Also confirm `k_axis` is wide enough: it must span from `k_min = -(Lh-1)` to `k_max = (Lx-1)+(Lh-1)` to accommodate all shift positions.
- **The overflow LED (D23) is lit.** This means $L_y = L_x + L_h - 1 > 7$, so the output sequence extends beyond the 7-LED display. All output values are still computed and transmitted serially—use the complete `y_hw` vector in MATLAB for full verification. The LED display shows only $y[0]$–$y[6]$.

- **Animation is too fast to observe the sliding kernel.** Increase the `pause()` duration in the loop (e.g., `pause(1.2)` for slower playback). When recording, use a screen capture tool that records at a minimum of 15 frames per second to ensure the video is viewable.

Appendix C: Arduino Reference Templates

```
// ---- Pin definitions (Mega full map; no pin conflicts) ----
const int DIP_PINS[8] = {2, 3, 4, 5, 6, 7, 8, 9};
const int BTN_PINS[4] = {10, 11, 12, 13};
const int LED_PINS[7] = {A0, A1, A2, A3, A4, A5, 22};
const int LED_OVF_PIN = 23;

const int LH_MAX = 8;
const int LX_MAX = 4;

int h[LH_MAX];
int x_in[LX_MAX];
int y_out[LH_MAX + LX_MAX - 1];
int Ly;

void readDIPSwitches() {
    for (int i = 0; i < LH_MAX; i++) {
        h[i] = digitalRead(DIP_PINS[i]);
    }
}

void readPushbuttons() {
    for (int i = 0; i < LX_MAX; i++) {
        x_in[i] = digitalRead(BTN_PINS[i]);
    }
}

void computeConvolution(int lh_actual, int lx_actual) {
    Ly = lh_actual + lx_actual - 1;
    for (int n = 0; n < Ly; n++) y_out[n] = 0;

    for (int n = 0; n < Ly; n++) {
        for (int k = 0; k < lx_actual; k++) {
            int hk = n - k;
            if (hk >= 0 && hk < lh_actual) {
                y_out[n] += x_in[k] * h[hk];
            }
        }
    }
}

void displayOutput() {
    for (int n = 0; n < 7 && n < Ly; n++) {
```

```
        digitalWrite(LED_PINS[n], (y_out[n] > 0) ? HIGH : LOW);
    }

    bool overflow = false;
    for (int n = 7; n < Ly; n++) {
        if (y_out[n] != 0) overflow = true;
    }
    digitalWrite(LED_OVF_PIN, overflow ? HIGH : LOW);
}

void sendSerialData(int lh_actual, int lx_actual) {
    Serial.print("H:");
    for (int i = 0; i < lh_actual; i++) {
        Serial.print(h[i]);
        if (i < lh_actual - 1) Serial.print(",");
    }
    Serial.print(";X:");
    for (int i = 0; i < lx_actual; i++) {
        Serial.print(x_in[i]);
        if (i < lx_actual - 1) Serial.print(",");
    }
    Serial.print(";Y:");
    for (int i = 0; i < Ly; i++) {
        Serial.print(y_out[i]);
        if (i < Ly - 1) Serial.print(",");
    }
    Serial.println();
}
```

Appendix D: MATLAB Reference Templates

```
% --- Serial setup (skeleton-compatible) ---
port = 'COM3';
baud = 115200;

s = serialport(port, baud);
configureTerminator(s, "CR/LF");
flush(s);
pause(2);

raw_line = readline(s);
parts    = strsplit(raw_line, ';');

h_str = strsplit(erase(parts{1}, 'H:'), ',');
x_str = strsplit(erase(parts{2}, 'X:'), ',');
y_str = strsplit(erase(parts{3}, 'Y:'), ',');

h_hw = str2double(h_str);
```

```
x_hw = str2double(x_str);
y_hw = str2double(y_str);

% --- Task 1: verification ---
y_matlab = conv(x_hw, h_hw);
max_err = max(abs(y_hw - y_matlab));

fprintf('=== Task 1: Hardware vs. MATLAB Verification ===\n');
fprintf('h[n]     = %s\n', num2str(h_hw));
fprintf('x[n]     = %s\n', num2str(x_hw));
fprintf('y_matlab = %s\n', num2str(y_matlab));
fprintf('y_hw     = %s\n', num2str(y_hw));
fprintf('Max abs error: %d\n', max_err);
if max_err == 0
    fprintf('RESULT: PASS -- hardware matches MATLAB exactly.\n');
else
    fprintf('RESULT: FAIL -- discrepancy detected.\n');
end

% --- Task 2: 1x3 stem plot ---
figure('Position', [100 100 1100 380]);

subplot(1, 3, 1);
n_x = 0 : length(x_hw)-1;
stem(n_x, x_hw, 'filled', 'b');
xlabel('n'); ylabel('x[n]'); title('Input x[n]'); grid on;

subplot(1, 3, 2);
n_h = 0 : length(h_hw)-1;
stem(n_h, h_hw, 'filled', 'b');
xlabel('n'); ylabel('h[n]'); title('Impulse Response h[n]'); grid on;

subplot(1, 3, 3);
n_y = 0 : length(y_matlab)-1;
stem(n_y, y_matlab, 'b', 'filled', 'DisplayName', 'MATLAB conv()');
hold on;
stem(n_y + 0.15, y_hw, 'r--', 'DisplayName', 'Arduino hardware');
xlabel('n'); ylabel('y[n]'); title('Output y[n] = x[n] * h[n]');
legend('Location', 'best'); grid on;

sgtitle(sprintf('Convolution Machine --- Lab 4 | %s | ID: %s', ...
        'YOUR NAME', 'YOUR STUDENT ID'));

% --- Task 3: animation skeleton ---
Lx = length(x_hw); Lh = length(h_hw); Ly_v = Lx + Lh - 1;
k_min = -(Lh - 1); k_max = (Lx - 1) + (Lh - 1); k_axis = k_min:k_max;
Nk = length(k_axis);
```

```
x_ext = zeros(1, Nk);
for kk = 0:Lx-1
    idx = find(k_axis == kk);
    x_ext(idx) = x_hw(kk + 1);
end

h_flip = fliplr(h_hw);
figure('Name', 'Convolution Animation', 'Position', [50 50 960 600]);

for n0 = 0:Ly_v-1
    clf;
    h_shifted = zeros(1, Nk);
    for ii = 0:Lh-1
        k_val = n0 - ii;
        idx = find(k_axis == k_val);
        if ~isempty(idx), h_shifted(idx) = h_flip(ii + 1); end
    end

    subplot(2, 1, 1);
    stem(k_axis, x_ext, 'b', 'filled', 'DisplayName', 'x[k]'); hold on;
    stem(k_axis, h_shifted, 'r', 'LineWidth', 2, ...
         'DisplayName', sprintf('h[%d-k]', n0));
    title(sprintf('Frame n_0 = %d : y[%d] = %.0f', n0, n0, y_matlab(n0+1)));
    xlabel('k'); ylabel('Amplitude'); legend('Location', 'best');
    grid on; xlim([k_min-0.5, k_max+0.5]);

    subplot(2, 1, 2);
    n_out = 0:n0;
    stem(n_out, y_matlab(1:n0+1), 'g', 'filled', 'LineWidth', 1.8);
    xlabel('n'); ylabel('y[n]'); title('Output y[n] (accumulated so far)');
    xlim([-0.5, Ly_v - 0.5]); ylim([0, max(y_matlab) + 1]); grid on;

    drawnow;
    pause(0.6);
end
```

References and Inspiration

This laboratory brings the central operation of signal processing—convolution—from abstract mathematics to concrete hardware and audible reality.

University Course Inspirations

Rice University ELEC 241: Real-Time Convolution on Embedded Systems

Lab 04 is inspired by Rice University's flagship course on real-time signal processing, which emphasizes implementing convolution directly on resource-constrained microcontrollers. Rice's philosophy is that understanding convolution requires building it yourself—hand-computing small convolutions, then watching the Arduino perform thousands of convolutions per second. This lab follows Rice's approach of making the abstract mathematical operation viscerally real through code, hardware measurements, and audible results.

MIT 6.003: Linear Time-Invariant Systems and Convolution

MIT's treatment of convolution as the fundamental operation linking system impulse response to arbitrary input-output behavior provides the theoretical foundation. MIT's emphasis on understanding convolution geometrically (flipping, shifting, multiplying, integrating) informs the hand-calculation exercises.

Georgia Tech ECE 2026: Convolution in Hardware Audio Processing

Georgia Tech's real-time audio DSP course uses convolution for audio effects (reverb, filtering) and emphasizes the connection between mathematical convolution and perceptual audio quality. This lab's focus on convolution for audio effects (impulse response convolution reverb) mirrors Georgia Tech's approach.

MIT 6.007: System Response and Convolution

The relationship between system impulse response and convolution, and the verification of convolution through measured system outputs, reflect MIT 6.007's rigor in connecting circuit behavior to signal processing theory.

Duke University ECE 280: Lab Modernization

This lab is part of Duke's curriculum redesign emphasizing that convolution is not an abstract mathematical concept but the core operation underlying all filtering, signal processing, and system analysis.

References

- A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010. Chapter 2: Convolution and LTI systems.
- A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed. Pearson, 1997. Chapter 2: Convolution.
- J. H. McClellan, R. W. Schafer, and M. A. Yoder, *DSP First: A Multimedia Approach*, 2nd ed. Pearson, 2015. Chapters on convolution and filtering.
- S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, 2nd ed. California Technical Publishing, 1997. Chapter 6: Convolution.